GENPACT

GigaAcademy Internship

Designing & Implementing an SOD System

Intern: Diellza Raçi

Stream: Artificial Intelligence

Mentor: Berat Ujkani, PhD

Capability Lead: Blerona Ibrahimi

May, 2026

ABSTRACT

This project presents the design and implementation of a Salient Object Detection (SOD) system using deep learning techniques and convolutional neural networks (CNNs). The goal of salient object detection is to identify and segment the most visually important object or region within an image. Unlike traditional object detection approaches that attempt to detect all objects, SOD focuses specifically on the dominant foreground object that naturally attracts human attention.

The system was implemented from scratch using PyTorch and trained on the DUTS dataset, which contains thousands of images paired with pixel-level saliency masks. The project included all major stages of an end-to-end machine learning pipeline, including dataset preprocessing, normalization, data augmentation, model design, training, evaluation, experimentation, and visualization.

A CNN encoder–decoder architecture was developed to generate saliency masks from RGB input images. The encoder extracted hierarchical image features using convolutional layers, batch normalization, ReLU activations, and max pooling operations, while the decoder reconstructed the final segmentation mask using transposed convolution layers. The model was trained using a combined Binary Cross Entropy (BCE) and Intersection-over-Union (IoU) loss function optimized with the Adam optimizer.

The system was evaluated using IoU, Precision, Recall, F1-score, and Mean Absolute Error (MAE). Experimental analysis was also conducted by modifying learning rates and introducing dropout regularization to study their impact on segmentation performance. The baseline model achieved the best overall performance and produced accurate saliency masks for a variety of image types and scenes.

In addition to the training and evaluation pipeline, an interactive Gradio-based demo application was developed, allowing users to upload custom images and visualize predicted saliency masks and overlay outputs in real time. The project successfully demonstrates the implementation of a complete salient object detection workflow and strengthens understanding of deep learning, image segmentation, and computer vision systems.

Table of contents

1. INTRODUCTION 4
2. DATASET 4
   2.1. DUTS Dataset 4
   2.2. Dataset Inspection 5
3. DATA PREPROCESSING 5
4. MODEL ARCHITECTURE 6
5. LOSS FUNCTION AND OPTIMIZATION 6
6. TRAINING PROCESS 7
7. EVALUATION METRICS 7
8. EXPERIMENTAL RESULTS 8
   8.1. Baseline Model 8
   8.2. Experiment 1 – Lower Learning Rate 8
   8.3. Experiment 2 – Dropout Regularization 8
   8.4. Comparison Table 8
9. VISUALIZATION RESULTS 8
10. DEMO SYSTEM 9
11. CHALLENGES ENCOUNTERED 9
12. CONCLUSION 10

REFERENCES 10

1. INTRODUCTION

Salient Object Detection (SOD) is a computer vision task focused on identifying the most visually important object or region in an image. Unlike traditional object detection systems, which aim to classify and localize every object within an image, SOD concentrates on highlighting the dominant region that naturally attracts human attention.

The goal of this project was to design and implement an end-to-end SOD system using convolutional neural networks (CNNs). The project included dataset preprocessing, model development, training, evaluation, experimentation, visualization, and an interactive demo application.

The system was implemented using PyTorch and trained on the DUTS dataset in Google Colab.

2. DATASET

2.1. DUTS Dataset

The DUTS dataset was used for training and evaluating the salient object detection model. It is a widely used benchmark dataset containing natural images paired with pixel-level saliency masks. Each mask highlights the dominant foreground object within the image.

Project dataset statistics:

Dataset Split	Number of Images
DUTS-TR Images	10,553
DUTS-TE Images	5,019

The dataset includes a variety of scenes, objects, and backgrounds, making it suitable for evaluating general-purpose segmentation models.

2.2. Dataset Inspection

Before training, random images and masks were visualized to verify image quality, mask alignment, and dataset diversity.

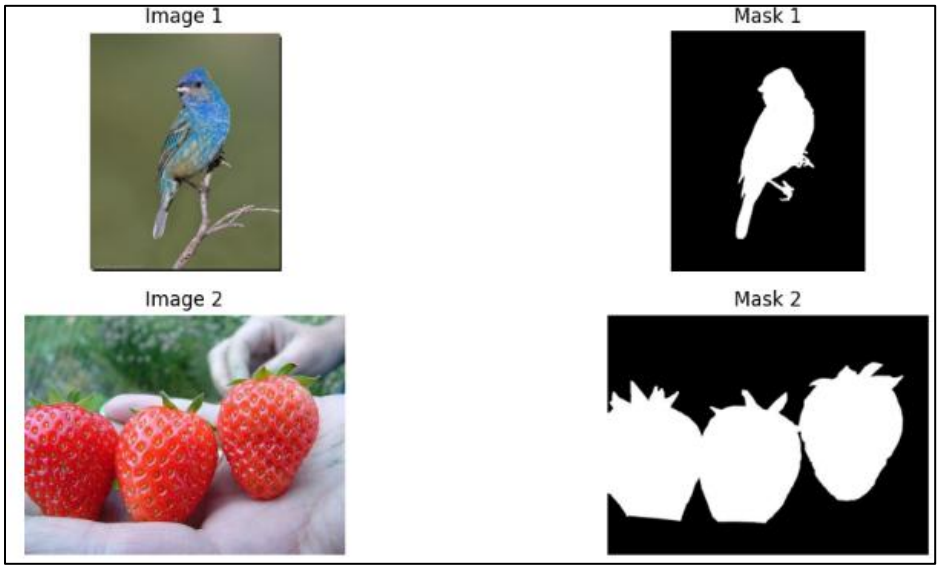

Figure 1. Dataset Inspection - Images & Masks

3. DATA PREPROCESSING

All images and masks were resized to 224 × 224 pixels and normalized to the range 0–1 before training.

The dataset was divided using the following split:

Split	Percentage
Training	70%
Validation	15%
Testing	15%

To improve generalization and reduce overfitting, augmentations such as horizontal flipping and brightness variation were applied during training.

4. MODEL ARCHITECTURE

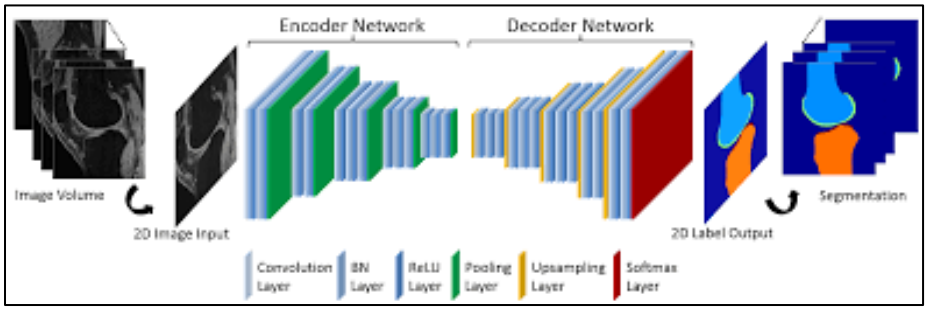

Figure 2. CNN Model Architecture

The implemented model follows a CNN encoder–decoder architecture designed for image segmentation tasks. The encoder extracts important spatial and semantic features from the input image, while the decoder reconstructs these features into a pixel-level saliency mask.

The encoder consists of four convolutional blocks with batch normalization, ReLU activation functions, and max pooling layers. As the image passes through the encoder, spatial dimensions are reduced while feature depth increases, allowing the network to learn higher-level image representations.

The decoder uses transposed convolution layers to gradually upsample the feature maps back to the original image resolution. The final layer produces a single-channel saliency mask using a sigmoid activation function.

The architecture was intentionally kept lightweight to allow efficient training in Google Colab while still achieving good segmentation performance.

5. LOSS FUNCTION AND OPTIMIZATION

The model was trained using a combination of Binary Cross Entropy (BCE) loss and IoU-based loss. BCE measures pixel-level prediction accuracy, while IoU improves overlap quality between predicted and ground-truth masks.

The Adam optimizer was used with a learning rate of 1e-3 because of its stable and efficient convergence behavior. Early stopping was implemented to prevent overfitting, and checkpoints were saved during training to preserve the best-performing model and allow training recovery if interrupted.

6. TRAINING PROCESS

The model was trained using Google Colab with GPU acceleration.

The training loop included forward propagation, loss computation, backpropagation, parameter optimization, validation after every epoch, and metric logging.

Training hyperparameters:

Split	Value
Image Size	224 x 224
Batch Size	16
Epochs	25
Optimizer	Adam
Learning Rate	1e-3

The best-performing model was automatically saved based on validation loss.

7. EVALUATION METRICS

The model was evaluated using IoU, Precision, Recall, F1-score, and Mean Absolute Error (MAE). These metrics were used to measure segmentation quality, prediction accuracy, and overlap between predicted masks and ground-truth masks.

Higher IoU, Precision, Recall, and F1-score values indicate better performance, while lower MAE values indicate lower prediction error.

8. EXPERIMENTAL RESULTS

8.1. Baseline Model

The baseline model used a learning rate of 1e-3 without dropout regularization and achieved the best overall performance.

8.2. Experiment 1 – Lower Learning Rate

The first experiment reduced the learning rate to 1e-4. The results remained very similar to the baseline model.

8.3. Experiment 2 – Dropout Regularization

The second experiment added dropout layers to the encoder. This configuration performed worse than the baseline model and caused underfitting.

8.4. Comparison Table

Model	IoU	Precision	Recall	F1-score	MAE
Baseline	0.5615	0.7124	0.7290	0.7159	0.2001
Experiment 1	0.5609	0.7118	0.7288	0.7154	0.2001
Experiment 2	0.1926	0.7308	0.2083	0.3157	0.2778

9. VISUALIZATION RESULTS

The model successfully generated saliency masks and overlay visualizations for various images. Visual inspection showed that the system could correctly identify dominant foreground objects in many scenes.

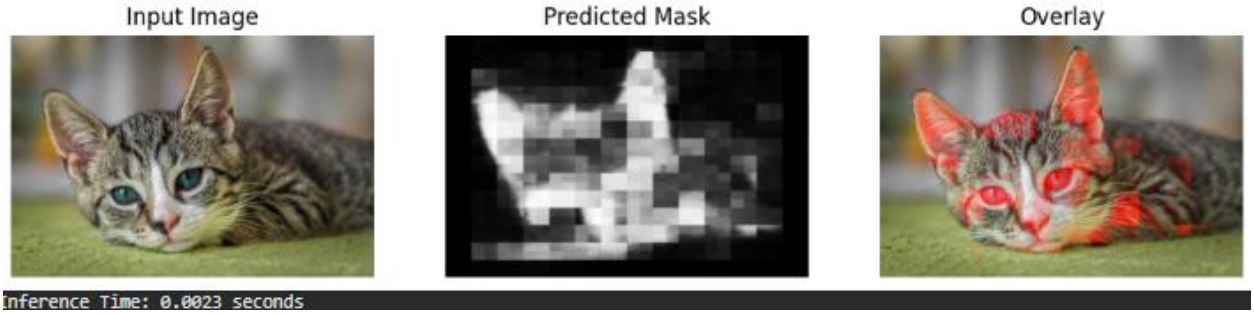

Figure 3. Predicted Mask, Overlay & Inference Time of Input Image

10. DEMO SYSTEM

A Gradio-based demo application was developed to provide an interactive interface for testing the trained model. Users can upload custom images and generate predicted saliency masks together with overlay visualizations.

The demo also displays inference time, allowing users to evaluate the model's prediction speed and overall segmentation quality in real time.

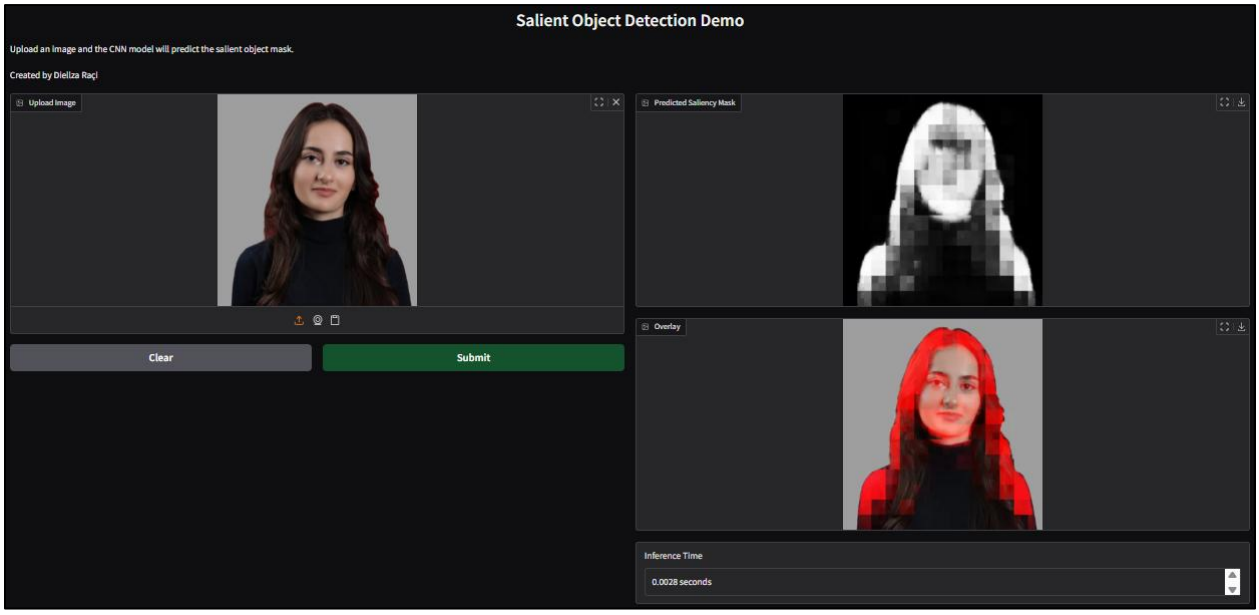

Figure 4. SOD System Demo

11. CHALLENGES ENCOUNTERED

Several challenges were encountered during development, including GPU memory limitations, balancing image resolution with training speed, and maintaining compatibility between different experimental model configurations. Additional effort was required to prevent overfitting while keeping the architecture lightweight enough for efficient training in Google Colab.

12. CONCLUSION

This project successfully implemented a complete salient object detection pipeline using convolutional neural networks. The system included preprocessing, augmentation, training, evaluation, experimentation, visualization, and an interactive Gradio demo.

The baseline encoder–decoder architecture achieved the best overall performance and demonstrated that relatively simple CNN-based segmentation models can effectively detect salient objects in images. The experiments also showed how architectural and hyperparameter changes influence segmentation performance.

Overall, the project strengthened understanding of deep learning workflows, image segmentation, model evaluation, and practical computer vision development.

REFERENCES

1. DUTS Dataset - https://www.kaggle.com/datasets/balraj98/duts-saliency-detection-dataset
2. PyTorch Documentation - https://pytorch.org/
3. Gradio Documentation - https://www.gradio.app/
4. Udemy Machine Learning Course - https://genpact.udemy.com/course/machinelearning/